



# Fisiere : Cuprins

- 1. Operatii cu fisiere. Clasa File**
- 2. Acces aleator la fisiere**
- 3. Fluxuri de date**
- 4. Serializarea obiectelor**

# Serializarea obiectelor

- **Serializarea** este posibilitatea de a citi si scrie obiecte din memorie in fluxuri de date. Serializarea este transformarea unui obiect într-o secvență de octeți, din care să poată fi refăcut ulterior obiectul original.
- Pentru a putea serializa un obiect, clasa din care face parte trebuie sa implementeze interfata **Serializable**
- Sunt serializate doar **atributele de instanta**, nu se serializeaza atributele statice si metodele

# Serializarea obiectelor

- Nu toate obiectele pot fi serializate (ex. fluxuri, conexiuni retea)
- Serializarea se aplica **recursiv**, la salvarea unui obiect se serializeaza toate obiectele cu care este asociat (toate attributele de tip obiect serializabil)
- Ex: daca serializam un obiect Examen se vor serializa toate obiectele Student continute

# Serializarea obiectelor

- Clasele folosite:
  - ObjectOutputStream
  - ObjectInputStream
- Mecanismul implicit va salva:
  - Numele clasei obiectului.
  - Signatura clasei.
  - Valorile tuturor câmpurile serializabile.
- Acesta este un proces recursiv, de serializare a obiectelor referite ("rețeaua de obiecte");
- Metode:
  - writeObject, pentru scriere și
  - readObject, pentru restaurare.
- Pentru a putea fi serializat un obiect trebuie sa fie instanta a unei clase care implementeaza una din interfetele :
  - - java.io.Serializable sau
  - - java.io.Externalizable (care extinde clasa Serializable)

# Serializarea obiectelor

- Interfata Serializable nu are nici o metoda, ea da doar posibilitatea de a specifica faptul ca se doreste ca o anumita clasa sa poata fi serializata. Declaratia unei astfel de clase ar fi :
- `class ClasaSerializabila implements Serializable {...}`
- In urma serializarii obiectele pot fi salvate într-un fisier, în acelasi fisier putând fi salvate si mai multe obiecte. Operatiile de intrare iesire la nivelul obiectelor se realizeaza prin intermediul unor fluxuri de obiecte, implementate de clasele `ObjectInputStream` si `ObjectOutputStream`.
- Salvarea unui obiect într-un fisier se realizeaza astfel :
  - `MyObject o = new MyObject();`
  - `FileOutputStream fout = new FileOutputStream("fisier");`
  - `ObjectOutputStream sout = new ObjectOutputStream(fout);`
  - `sout.writeObject(o);`
- Restaurarea unui obiect salvat într-un fisier se face într-o maniera asemanatoare:
  - `FileInputStream fin = new FileInputStream("fisier");`
  - `ObjectInputStream sin = new ObjectInputStream(fin);`
  - `o = (MyObject) sin.readObject();`

# Serializarea obiectelor

- Pe lângă metodele de scriere/citire a obiectelor cele două clase pun la dispoziție și metode pentru scrierea tipurilor de date primare, astfel încât apeluri ca cele de mai jos sunt permise :
  - `FileOutputStream ostream = new FileOutputStream("t.tmp");`
  - `ObjectOutputStream p = new ObjectOutputStream(ostream);`
  - `p.writeInt(12345);`
  - `p.writeObject("Today");`
  - `p.writeObject(new Date());`
  - `p.flush();`
  - `ostream.close();`
  - `FileInputStream istream = new FileInputStream("t.tmp");`
  - `ObjectInputStream p = new ObjectInputStream(istream);`
  - `int i = p.readInt();`
  - `String today = (String)p.readObject();`
  - `Date date = (Date)p.readObject();`
  - `istream.close();`

# Serializarea obiectelor

- `ObjectInputStream` si `ObjectOutputStream` implementeaza indirect interfetele `DataInput`, respectiv `DataOutput`, interfețe ce declara metode atât pentru scrierea/citirea datelor primitive, cât si pentru scrierea/citirea obiectelor.
- Pentru transferul obiectelor sunt folosite metodele:
  - `final void writeObject( java.lang.Object obj) throws java.io.IOException`
  - `final java.lang.Object readObject( ) throws java.io.OptionalDataException, java.lang.ClassNotFoundException, java.io.IOException`
- Acestea apeleaza la rândul lor metodele implicite de transfer `defaultWriteObject` si `defaultReadObject` (având aceleasi semnături ca mai sus)

# Clasa ObjectOutputStream

## ■ Constructor

- `public ObjectOutputStream( java.io.OutputStream out )` throws `java.io.IOException`

## ■ Metode

- `void close( )` throws `java.io.IOException`
- `final void defaultWriteObject( )` throws `java.io.IOException`
- `void flush( )` throws `java.io.IOException`
- `void reset( )` throws `java.io.IOException`
- `void write( bytest b )` throws `java.io.IOException`
- `void write( bytest b, int off, int len )` throws `java.io.IOException`
- `void write( int data )` throws `java.io.IOException`
- `void writeBoolean( boolean data )` throws `java.io.IOException`
- `void writeByte( int data )` throws `java.io.IOException`
- `void writeBytes( java.lang.String data )` throws `java.io.IOException`
- `void writeChar( int data )` throws `java.io.IOException`
- `void writeChars( java.lang.String data )` throws `java.io.IOException`
- `void writeDouble( double data )` throws `java.io.IOException`
- `void writeFloat( float data )` throws `java.io.IOException`
- `void writeInt( int data )` throws `java.io.IOException`
- `void writeLong( long data )` throws `java.io.IOException`
- `final void writeObject( java.lang.Object obj )` throws `java.io.IOException`
- `void writeShort( int data )` throws `java.io.IOException`
- `void writeUTF( java.lang.String data )` throws `java.io.IOException`



# Clasa ObjectInputStream

## ■ Constructor

- `public ObjectInputStream( java.io.InputStream in )` throws `java.io.IOException`, `java.io.StreamCorruptedException`

## ■ Metode

- `int available( )` throws `java.io.IOException`
- `void close( )` throws `java.io.IOException`
- `final void defaultReadObject( )` throws `java.io.IOException`, `java.lang.ClassNotFoundException`, `java.io.NotActiveException`
- `int read( bytest data, int offset, int length )` throws `java.io.IOException`
- `int read( )` throws `java.io.IOException`
- `boolean readBoolean( )` throws `java.io.IOException`
- `byte readByte( )` throws `java.io.IOException`
- `char readChar( )` throws `java.io.IOException`
- `double readDouble( )` throws `java.io.IOException`
- `float readFloat( )` throws `java.io.IOException`

# Clasa ObjectInputStream

- ❑ void readFully( bytest data ) throws java.io.IOException
- ❑ void readFully( bytest data, int offset, int size ) throws java.io.IOException
- ❑ int readInt( ) throws java.io.IOException
- ❑ java.lang.String readLine( ) throws java.io.IOException
- ❑ long readLong( ) throws java.io.IOException
- ❑ final java.lang.Object readObject( ) throws java.io.OptionalDataException, java.lang.ClassNotFoundException, java.io.IOException
- ❑ short readShort( ) throws java.io.IOException
- ❑ int readUnsignedByte( ) throws java.io.IOException
- ❑ int readUnsignedShort( ) throws java.io.IOException
- ❑ java.lang.String readUTF( ) throws java.io.IOException
- ❑ synchronized void registerValidation( java.io.ObjectInputValidation obj, int prio ) throws java.io.NotActiveException, java.io.InvalidObjectException
- ❑ int skipBytes( int len ) throws java.io.IOException

# Serializarea obiectelor

...

```
Student s1 = new Student("Ion", 23);
```

```
try {
```

```
    FileOutputStream fos = new  
    FileOutputStream("student.dat");
```

```
    ObjectOutputStream oos = new  
    ObjectOutputStream(fos);
```

```
    oos.writeObject(s1);
```

```
    oos.close();
```

```
} catch (IOException e) {
```

```
    System.out.println("eroare");
```

```
}
```

# Deserializarea obiectelor

```
Student s1;  
try {  
    FileInputStream fis = new  
        FileInputStream("student.dat");  
    ObjectInputStream ois = new  
        ObjectInputStream(fis);  
    s1 = (Student) ois.readObject();  
    ois.close();  
} catch (IOException e) {  
    System.out.println("eroare");  
} catch (ClassNotFoundException e) {  
    System.out.println("eroare");  
}
```

# Cuvântul cheie transient

Pentru ca un anumit câmp sa nu fie salvat în urma serializarii, acesta trebuie declarat cu modificatorul transient. Aceste câmpuri vor fi ignorate de metodele writeObject si readObject.

Ex:

```
private transient x; → ignorat la serializare
```

Exemplu:

```
class MyObject implements Serializable {
```

```
    int x;
```

```
    private transient int y;
```

```
    public MyObject(int x, int y) {
```

```
        this.x = x;
```

```
        this.y = y;
```

```
    }
```

```
    public String toString() {
```

```
        return new String("x=" + x + ", y="+y);
```

```
    }
```

```
}
```

```
import java.io.*;
```

```
public class TestSerial {
```

```
    static MyObject obj;
```

```
    public static void main(String [] arg) {
```

```
        obj = new MyObject(10,20);
```

# Cuvântul cheie transient

```
■ try { FileOutputStream fout = new  FileOutputStream("fisier.tmp");
■         ObjectOutputStream sout = new  ObjectOutputStream(fout);
■         sout.writeObject(obj);
■         sout.flush();
■         sout.close();
■         fout.close();
■         System.out.println("A fost salvat obiectul " + obj);
■     }
■     catch (IOException e) {}
■     System.out.println("Restauram...");
■ try { FileInputStream fin = new  FileInputStream("fisier.tmp");
■     ObjectInputStream sin = new ObjectInputStream(fin);
■     try { obj = (MyObject) sin.readObject();
■         }
■     catch (ClassNotFoundException e) {}
■     sin.close();
■     fin.close();
■     System.out.println("A fost restaurat obiectul " + obj);
■     }
■     catch (IOException e) {}
■     }
■ }
```

# Cuvântul cheie transient

- Rezultatul acestui program va fi :
- A fost salvat obiectul x=10, y=20
- Restauram...
- A fost restaurat obiectul x=10, y=0
- Obs. Atunci când o clasă serializabilă derivă dintr-o altă clasă, salvarea câmpurilor clasei părinte se va face doar dacă și aceasta este serializabilă. În caz contrar, subclasa trebuie să salveze explicit și câmpurile moștenite.

■ Ex1:

```
■      class Parinte implements Serializable {  
■          int x; ...  
■      }  
■      class Fiu extends Parinte implements Serializable {  
■          int y; ...  
■      }
```

■ La serializare se salvează atât x cât și y.

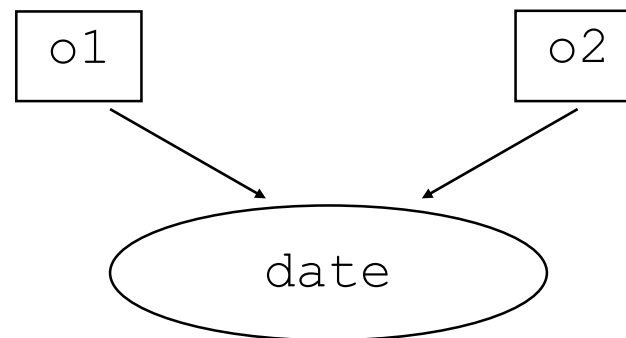
■ Ex1:

```
■      class Parinte {  
■          int x; ...  
■      }  
■      class Fiu extends Parinte implements Serializable {  
■          int y; ...  
■      }
```

■ Serializarea nu decurge normal.

# Folosirea serializării pentru copierea obiectelor

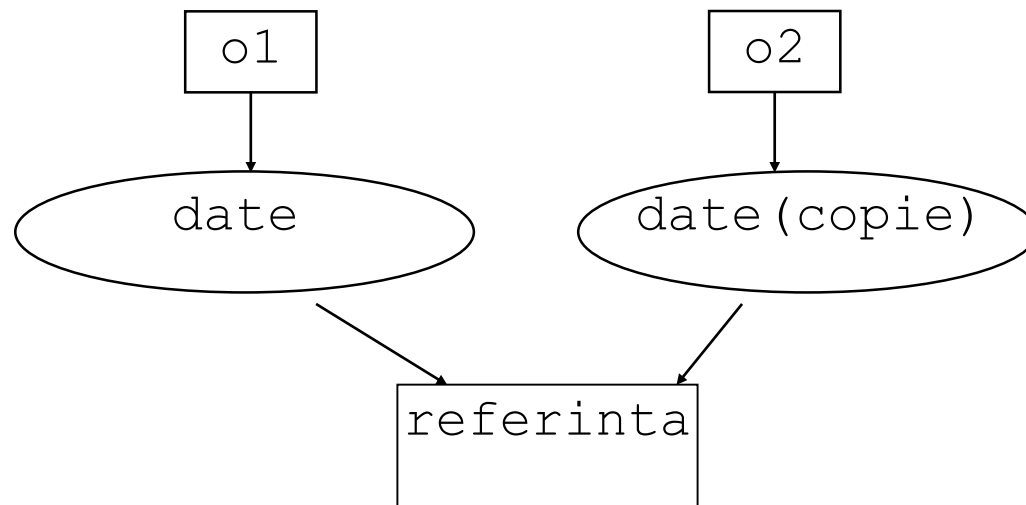
- Se știe că nu putem copia un obiect prin instrucțiunea de atribuire. O secvență de forma:
  - `MyObject o1 = new MyObject(10, 20);`
  - `MyObject o2 = o1;`
- nu face decât să declare obiectul `o2` ca fiind o referință la obiectul `o1`. Orice schimbare într-unul din cele două obiecte se va reflecta și în celălalt.





# Folosirea serializarii pentru copierea obiectelor

- O posibilitate de a face o copie unui obiect este folosirea metodei *clone()* a clasei *Object*.
- `MyObject o1 = new MyObject(10, 20);`
- `MyObject o2 = (MyObject) o1.clone();`



# Folosirea serializarii pentru copierea obiectelor

- Conversia la clasa `MyObject` este necesara deoarece metoda `clone()` returneaza un obiect de tip `Object`. Deficienta acestei metode este ca nu functioneaza corect decât atunci când clasa clonata nu are câmpuri referinta catre alte obiecte, obiectele referite nemaifiind copiate la rândul lor.
- O metoda `clone()` care sa realizeze o copie efectiva a unui obiect, împreuna cu copierea tuturor obiectelor referite de câmpurile acelui obiect poate fi implementata prin mecanismul serializarii astfel :
- ```
public Object clone() {
```
- ```
    try {
```
- ```
        ByteArrayOutputStream bout = new ByteArrayOutputStream();
```
- ```
        ObjectOutputStream out = new ObjectOutputStream(bout);
```
- ```
        out.writeObject(this);
```
- ```
        out.close();
```
- ```
        ByteArrayInputStream bin = new ByteArrayInputStream();
```
- ```
        ObjectInputStream in = new ObjectInputStream(bin);
```
- ```
        Object ret = in.readObject();
```
- ```
        in.close();
```
- ```
        return ret;
```
- ```
    }
```
- ```
    catch (Exception e) { System.out.println(e);
```
- ```
        return null;
```
- ```
    }
```
- ```
}
```



# Cuprins

- I. Notiuni de programare orientata obiect**
- II. Organizarea claselor in pachete. Clase fundamentale**
- III. Mostenirea claselor**
- IV. Clase abstracte si interfete**
- V. Polimorfismul**
- VI. Tratarea erorilor**
- VII. Lucrul cu fisiere**
- VIII. Colectii de obiecte**
- IX. Interfete grafice folosind AWT. Applet-uri**



# Colectii : Cuprins

1. **Ce sunt colectiile**
2. **Tehnologii folosite de colectii**
3. **Proprietati ale colectiilor**
4. **Interfete ale colectiilor**
5. **Clase ale colectiilor**
6. **Multimi (Set)**
7. **Liste**
8. **Asocieri (Map)**
9. **Iteratori**
10. **Operatii cu colectii**

# Ce sunt colectiile

- Colectiile se aseamana cu tablourile (array) in sensul ca un singur obiect este folosit pentru a reprezenta un **grup** de obiecte
- Colectiile sunt mult mai **flexibile** decat tablourile, ele pot creste dinamic, pot sorta obiectele, iar unele operatii sunt mai rapide
- Nu exista colectii de tipuri primitive ci doar **colectii de obiecte**



# Colectii : Cuprins

1. Ce sunt colectiile
2. **Tehnologii folosite de colectii**
3. Proprietati ale colectiilor
4. Interfete ale colectiilor
5. Clase ale colectiilor
6. Multimi (Set)
7. Liste
8. Asocieri (Map)
9. Iteratori
10. Operatii cu colectii

# Tehnologii de stocare

- Exista patru tehnologii folosite pentru stocarea obiectelor in memorie:
  - **Tablouri** (array)
  - **Liste inlantuite** (linked-lists)
  - **Arbori** (tree)
  - **Tabele de hashing** (hash table)
- Indiferent ce tehnologie se foloseste, operatiile pentru colectiile de obiecte sunt aceleasi, ele pot fi doar mai performante in functie de tehnologia de stocare

# Tablouri

## ■ Tablouri (array)

- ☐ Elementele sunt ordonate si pot exista duplicate
- ☐ Dimensiunea tabloului este fixa
- ☐ Este performant la accesarea datelor
- ☐ Este incet la adaugarea si stergerea obiectelor

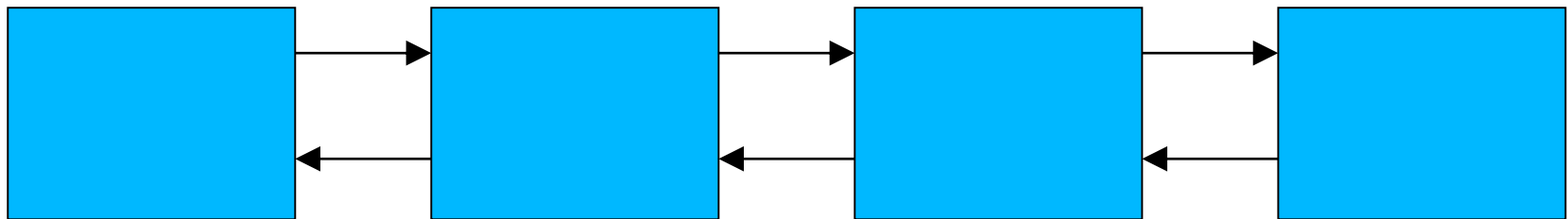




# Liste

## ■ Liste inlantuite (linked-lists)

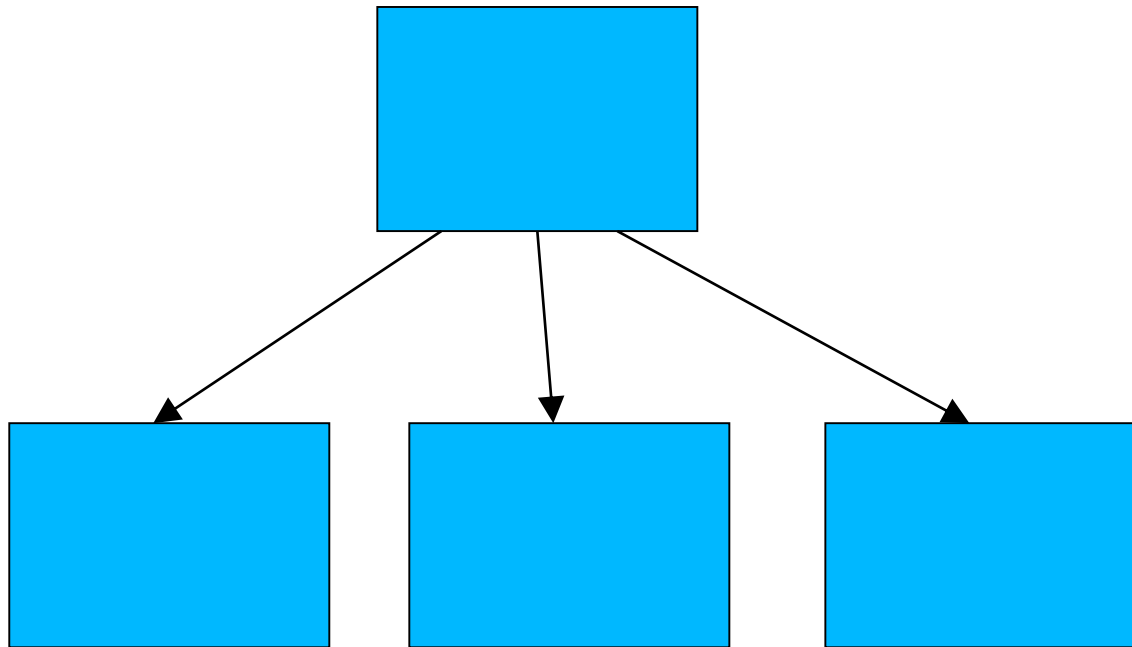
- Elementele sunt ordonate si pot exista duplicate
- Dimensiunea este variabila
- Este incet la accesarea datelor
- Adaugarea si stergerea obiectelor este rapida



# Arbori

## ■ Arbori (tree)

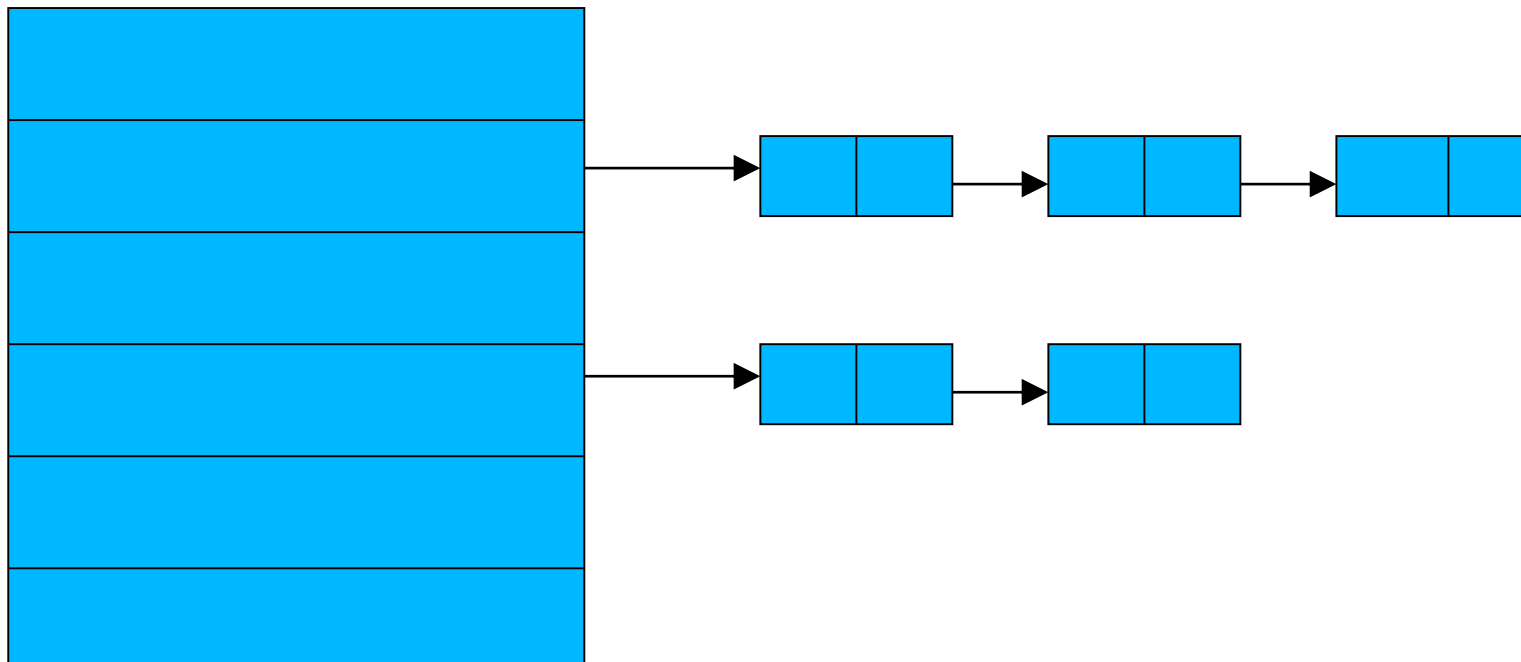
- Este o structura neliniara
- Datele sunt **sortate** si nu exista duplicate
- Dimensiunea este variabila



# Tabele de hashing

## ■ Tabele de hashing (hash table)

- Fiecare element este format dintr-o pereche cheie-valoare, cu cheie unica
- Dimensiunea este variabila
- Este performant la toate operatiile





# Colectii : Cuprins

1. Ce sunt colectiile
2. Tehnologii folosite de colectii
3. **Proprietati ale colectiilor**
4. Interfete ale colectiilor
5. Clase ale colectiilor
6. Multimi (Set)
7. Liste
8. Asocieri (Map)
9. Iteratori
10. Operatii cu colectii

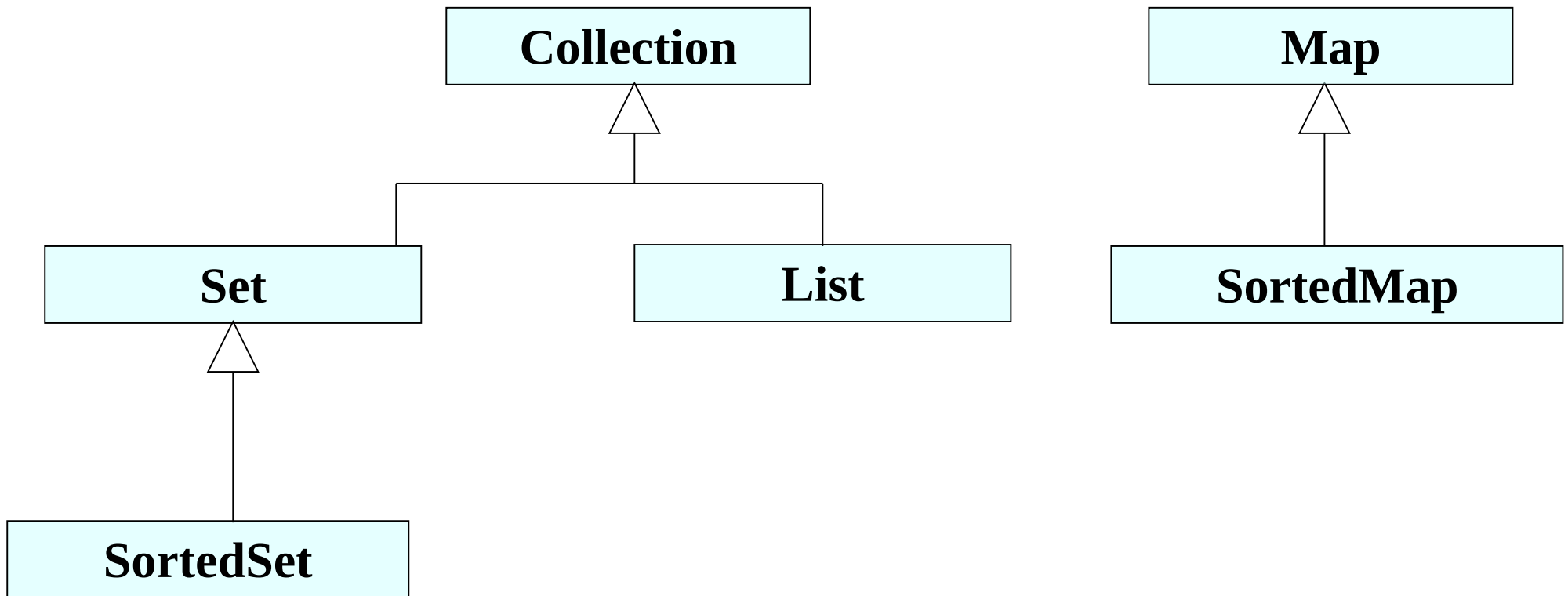
# Proprietati ale colectiilor

- Exista patru proprietati pe care le pot avea colectiile de obiecte:
  - **Sortate** – elementele sunt aranjate dupa un criteriu de tip mai mic/mai mare
  - **Ordonate** – se cunoaste pozitia fiecarui elemente
  - **Permit duplicate** – colectia poate contine doua elemente identice
  - **Folosesc chei** – se foloseste un obiect cheie pentru a identifica obiectul stocat

# Colectii : Cuprins

1. Ce sunt colectiile
2. Tehnologii folosite de colectii
3. Proprietati ale colectiilor
4. **Interfete ale colectiilor**
5. Clase ale colectiilor
6. Multimi (Set)
7. Liste
8. Asocieri (Map)
9. Iteratori
10. Operatii cu colectii

# Ierarhia interfetelor



# Interfete ale colectiilor

- **Collection** – este interfata generala pentru colectii, are operatii de adaugare, intergoare
- **Set** – multime, nu permite duplicate, nu este ordonata, nu este sortata, nu foloseste chei
- **List** – lista, este ordonata, nu este sortata, permite duplicate, nu foloseste chei
- **Map** – asociatie, foloseste chei unice, neordonat, nu permite duplicate



# Interfete ale colectiilor

- **Collection** pune la dispozitie urmatoarele metode:

```
boolean add(Object o) ;
```

```
void clear() ;
```

```
boolean contains(Object o) ;
```

```
boolean remove(Object o) ;
```

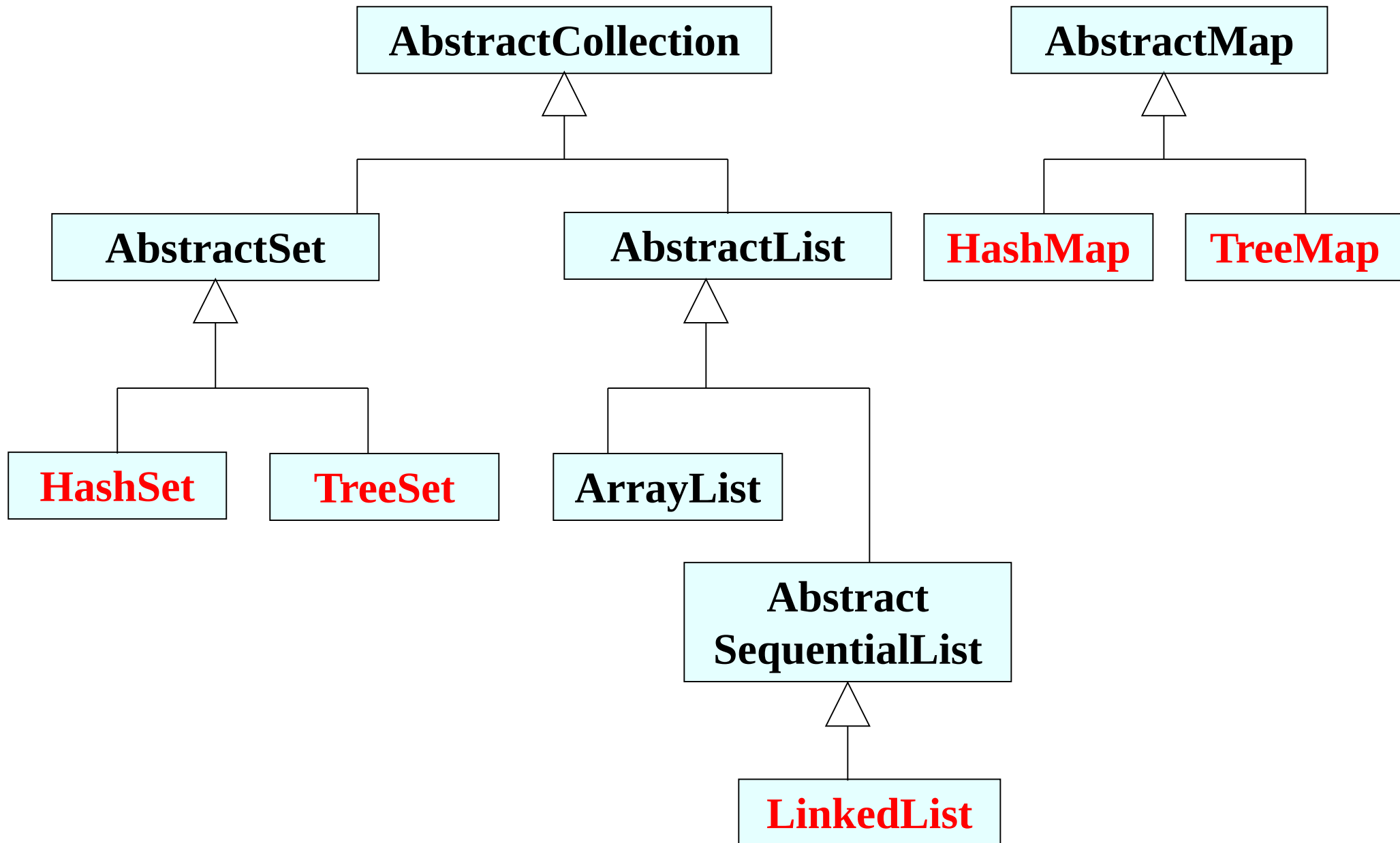
```
int size() ;
```



# Colectii : Cuprins

- 1. Ce sunt colectiile**
- 2. Tehnologii folosite de colectii**
- 3. Proprietati ale colectiilor**
- 4. Interfete ale colectiilor**
- 5. Clase ale colectiilor**
- 6. Multimi (Set)**
- 7. Liste**
- 8. Asocieri (Map)**
- 9. Iteratori**
- 10. Operatii cu colectii**

# Ierarhia claselor



# Colectii : Cuprins

- 1. Ce sunt colectiile**
- 2. Tehnologii folosite de colectii**
- 3. Proprietati ale colectiilor**
- 4. Interfete ale colectiilor**
- 5. Clase ale colectiilor**
- 6. Multimi (Set)**
- 7. Liste**
- 8. Asocieri (Map)**
- 9. Iteratori**
- 10. Operatii cu colectii**

# Multimi (set)

- Exista doua implementari importante de multime:
- **HashSet** – implementeaza interfata Set, nu este ordonata, nu este sortata si nu permite duplicate
- **TreeSet** – implementeaza interfata SortedSet, este sortata. Elementele trebuie sa poata fi comparate intre ele (elementele trebuie sa implementeze interfata **java.lang.Comparable**)



# Colectii : Cuprins

- 1. Ce sunt colectiile**
- 2. Tehnologii folosite de colectii**
- 3. Proprietati ale colectiilor**
- 4. Interfete ale colectiilor**
- 5. Clase ale colectiilor**
- 6. Multimi (Set)**
- 7. Liste**
- 8. Asocieri (Map)**
- 9. Iteratori**
- 10. Operatii cu colectii**

# Liste

- Exista trei implementari importante de lista:
  - **Vector** – este o implementare veche, nu se recomanda a mai fi folosita
  - **ArrayList** – implementeaza interfata List, dar foloseste tablouri, poate sa creasca dupa nevoie dar stergerea si inserarea in mijlocul listei sunt incete
  - **LinkedList** – implementeaza interfata List, folosind liste dublu inlantuite, poate fi parcursa in ambele directii



# Colectii : Cuprins

- 1. Ce sunt colectiile**
- 2. Tehnologii folosite de colectii**
- 3. Proprietati ale colectiilor**
- 4. Interfete ale colectiilor**
- 5. Clase ale colectiilor**
- 6. Multimi (Set)**
- 7. Liste**
- 8. Asocieri (Map)**
- 9. Iteratori**
- 10. Operatii cu colectii**



# Asocieri (map)

- Exista doua implementari importante de asocieri:
  - **HashMap** – implementeaza interfata Map, nu este ordonate, nu este sortata, nu permite duplicate, foloseste chei pentru gasirea rapida a obiectelor
  - **TreeMap** – implementeaza interfata SortedMap, este sortata, deci elementele trebuie sa implementeze interfata **java.lang.Comparable**

# Colectii : Cuprins

- 1. Ce sunt colectiile**
- 2. Tehnologii folosite de colectii**
- 3. Proprietati ale colectiilor**
- 4. Interfete ale colectiilor**
- 5. Clase ale colectiilor**
- 6. Multimi (Set)**
- 7. Liste**
- 8. Asocieri (Map)**
- 9. Iteratori**
- 10. Operatii cu colectii**

# Iteratori

- Parcurgerea colectiilor se face prin intermediul iteratorilor
- Un iterator implementeaza interfata **java.util.Iterator** si are functii pentru parcurgerea unor elemente:

```
public boolean hasNext() ;
```

```
public Object next() ;
```

```
public void remove() ;
```

# Iteratori

## ■ Exemplu:

```
Set m = new HashSet();
```

```
m.add("Ion");
```

```
m.add(new Integer(5));
```

```
Iterator i = m.iterator();
```

```
while(i.hasNext()) {
```

```
    System.out.println(i.next());
```

```
}
```



# Colectii : Cuprins

- 1. Ce sunt colectiile**
- 2. Tehnologii folosite de colectii**
- 3. Proprietati ale colectiilor**
- 4. Interfete ale colectiilor**
- 5. Clase ale colectiilor**
- 6. Multimi (Set)**
- 7. Liste**
- 8. Asocieri (Map)**
- 9. Iteratori**
- 10. Operatii cu colectii**

# Alte operatii cu colectii

- Clasa **java.util.Collections** pune la dispozitie metode statice pentru alte operatii cu colectii:

```
int binarySearch(List l, Object key) ;
```

```
void sort(List l) ;
```

```
void shuffle(List l) ;
```

```
void reverse(List l) ;
```

```
Object max(Collection c) ;
```